Using Object Detection to Classify Marine Animals

Group 5
Group Leader: Jonah Lee
Group Members: Siwen Liao, JJ Sutkus, and Jonah Lee
DS 4002
19 April 2026

Outline:

1. Project Details
2. Data Acquisition & Explanation
3. Analysis Plan & Justification
4. Tricky Analysis Decisions
5. Bias & Uncertainty Violation
6. Results & Conclusions
7. Next Steps
8. References

Project Details

Motivation:
Underwater images are difficult to examine due to blur, glare, occlusion, and similar-looking species. Reliable object detection matters for exhibit monitoring and conservation tooling. It is important to have empirical comparison when examining different architectures. Choosing the right model will affect things such as accuracy and computing cost.

Hypotheses:
Which object detection model (YOLO v4, Faster R-CNN, or SSD) performs best at classifying images of marine animals on COCO-style mAP and IoU (Intersection over Union)-thresholded behavior?

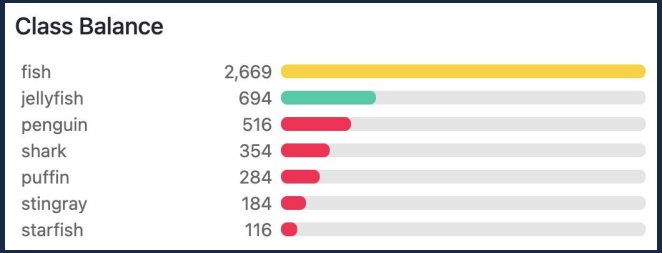
Research Question:
Which of these three models best balances detection quality and practical cost for classifying marine animals in our aquarium image set?

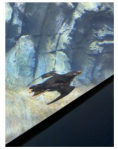
Modeling Approach:
To train/fine-tune three architecture on the same preprocessed splits, and evaluate on the same test images with comparable metrics

Goal:
To determine which model can best identify and classify marine animals in underwater images, measuring their success through metrics such as mean average precision (mAP), mAP at IoU 0.50 and 0.75, recall, and F1.

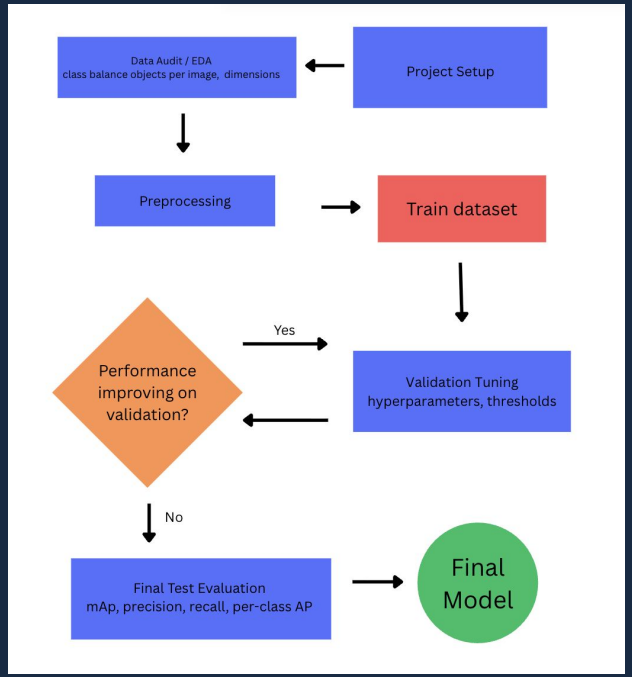
Data Acquisition

- Our data set includes 638 images of marine animals collected from the Henry Doorly Zoo in Omaha and the National Aquarium in Baltimore with no ethical or licensing concerns.
- Every object of interest is described by a bounding box and one of seven class labels: fish, jellyfish, penguin, puffin, shark, starfish, and stingray.
- Our image data was split into a training, validation, and test set with 448, 127, and 63 images, respectively.
- Stored as RGB images in common formats such as JPEG; labels appear in varying formats such as VOC-style XML and YOLO-style text files
- We cleaned our image data by standardizing the images, ensuring each image shares the same dimensions and lighting.
- Results may not generalize to wild marine environments.



Dataset Splits	Description	Potential Responses
train	Contains 448 images that will be utilized in training our three separate object detection models.	
valid	Contains 127 images that will be used to tune our three models by tuning hyperparameters and detecting overfitting.	
test	Contains 63 images that will be used to simulate the three models' performance in a real-world object detection scenario.	

Analysis Plan & Justification



Pipeline:

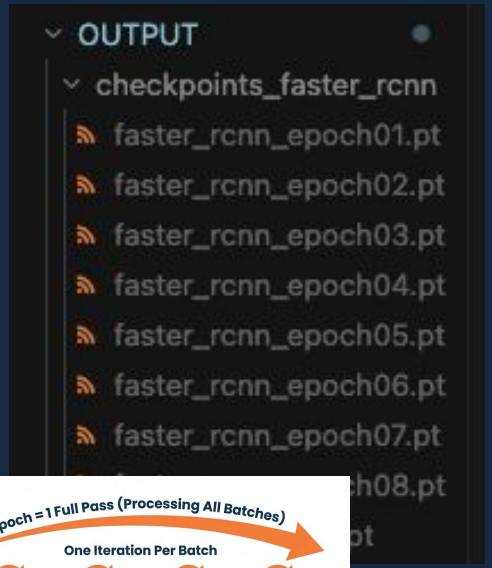
- ingesting raw images and annotations
- preprocessing them into unified splits and label formats suitable for each detector
- each model is fine-tuned separately on the training data with validation used for monitoring
- every model is evaluated on the same held-out test set, with metrics saved as JSON files for direct comparison
- each model begins from public pretrained weights and is then adapted to our seven marine classes on our data

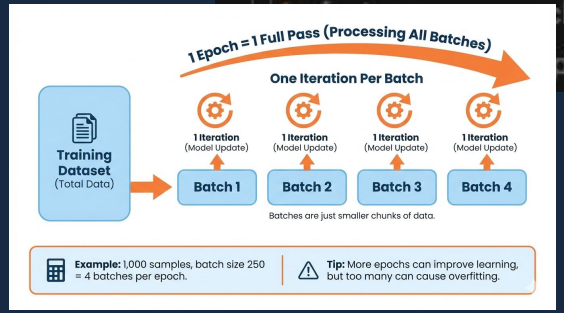
Challenges:

- maintaining multiple label formats
- managing long training runs on CPU for two-stage detectors
- aligning evaluation settings such as score thresholds and metric definitions across scripts

Tricky Analysis Decision

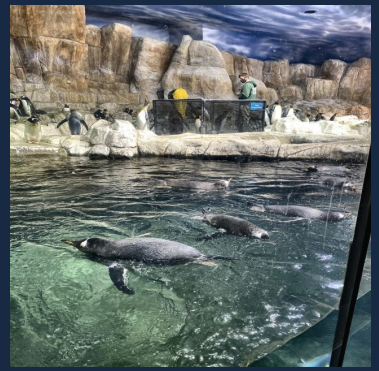
- Faster R-CNN training was interrupted before the eighth (final) epoch. Rather than restart from scratch, we resumed from the epoch-seven checkpoint, restored the optimizer state and ran an additional full epoch.
- We used a separate output directory to avoid overwriting earlier checkpoints, documenting that the final epoch was a continuation.
- Comparison to the other models remains fair
- Originally our plan required uninterrupted training, though computing limits led to this inconvenience

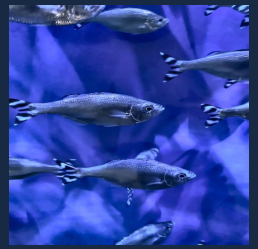




Bias & Uncertainty Validation

- data biased toward aquarium conditions
- lighting, background, species mix may not match open-water imagery
- class imbalance favoring certain species in both images and bounding boxes
- annotation noise from labeling affects all models, difficult to remove completely
- point estimates of mAP + other related metrics should be interpreted as indicative, not definitive due to test set size
- evaluated SSD with a lower score threshold than the other two models, complicating direct comparison of precision, recall and F1, so mAP was primary for cross-model ranking





Results & Conclusions

YOLO v4

- On the test set, achieved the highest mAP and mAP at IoU 0.50
- precision > recall, more conservative about calling a detection correct
- Low throughput on the machine, largest by parameter count

Faster R-CNN

- score second place in metrics, trailed behind YOLO v4 but still remained in a credible range for the same test images
- Similar evaluation speed to YOLO v4

SSD

- did not match the other two on accuracy
- several metrics were effectively zero, meaning that the model was not reliably detecting most categories
- reported the highest FPS and lowest latency and fewest parameters, meaning efficient but not competitive

Metric	YOLOv4 (Darknet)	Faster R-CNN (ResNet-50 FPN)	SSD (VGG-16)
COCO-style mAP	0.365	0.306	0.016
mAP@50	0.643	0.604	0.045
mAP@75	0.385	0.292	0.006
MAR@100	0.409	0.372	0.026
Micro F1 @ IoU 0.5	0.740	0.693	0.182
Micro precision @0.5	0.857	0.680	0.231
Micro recall @0.5	0.651	0.706	0.149
FPS (timed eval)	0.130	0.147	3.23
Latency (ms / image)	~7694	~6811	~309
Trainable parameters	64.0M	41.1M	24.5M

Next Steps

New Lines Of Exploration

- Test YOLOv5 and compare to YOLOv4 (YOLOv4 was generally considered more robust but YOLOv5 is smaller and more accessible)

Project Improvements

- Run multiple random splits instead of a designated train/val/test split and examine the variance of the evaluation metrics
- Use precision-recall curves to evaluate and compare the models to accommodate to different score thresholds

New Questions

- How does class imbalance affect the evaluation metrics results?

References

[1] Fritz. (n.d.). Object detection guide – Everything you need to know. Object detection guide – Everything you need to know

[2] K. Sarma et al., “A comparative study on Faster R-CNN, YOLO and SSD,” AIP Conference Proceedings, vol. 2971, 060044, 2024. Available: https://pubs.aip.org/aip/acp/article/2971/1/060044/3296342/A-comparative-study-on-faster-R-CNN-YOLO-and-SSD

[3] P. Tsirtsakis, G. Zacharis, G. S. Maraslidis, and G. F. Fragulis, “Deep learning for object recognition: A comprehensive review of models and algorithms,” International Journal of Cognitive Computing in Engineering, vol. 6, pp. 298–312, 2025.

[4] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards real-time object detection with region proposal networks,” IEEE Transactions on Pattern Analysis and Machine Intelligence, 2017.

[5] W. Liu et al., “SSD: Single Shot MultiBox Detector,” arXiv preprint arXiv:1905.016014, 2019.

Github Link: https://github.com/jonahsjlee/DS-4002_Group_Project3/

Thank you!